

Don’t Trust the Model’s Arithmetic: Deterministic Geometric Grounding for LLM/VLM-Driven Manipulation

Matteo Nini¹

Abstract—Large language model (LLM) and vision-language model (VLM) based robotic planners are increasingly validated in simulation, where an object’s position, size, and grasp geometry are either exactly known or scripted. We report on a real UR5cb manipulator deployment where these planners are routinely asked to compute grasp and release geometry themselves – release height, target stacking height, grasp width – and show that this model-computed arithmetic, not high-level task reasoning, is the dominant source of planning failure on physical hardware. Across six development sessions we diagnose a recurring family of geometry-grounding bugs (e.g. a systematic release-height overshoot from a top-vs-mid-body grasp mismatch, and a double-counted stacking height in a hybrid vision-language pipeline), each invisible to a symbolic or scripted simulator because none of these quantities are ever actually measured there. We fix five of these deterministically, removing geometric arithmetic from the model entirely and computing it from paired pick/release actions and measured depth; the sixth, a multi-object placement collision that recurs when sequential releases share a target zone, we diagnose but do not yet close, and report as the dominant residual failure. On a 120-trial real-hardware benchmark comparing an LLM planner against a camera-grounded VLM planner using a single unified model for both, we show this residual failure distribution occurs in both arms regardless of grounding modality, and argue that symbolic-simulation benchmarks of LLM-based robot planners – including a directly comparable prior taxonomy-driven study – structurally cannot observe this failure class, and that a geometry-grounding audit belongs in the evaluation protocol for any LLM/VLM planner headed to real hardware.

I. INTRODUCTION

Large language models (LLMs) and vision-language models (VLMs) are increasingly used as high-level planners for robotic manipulation, translating a natural-language instruction and a scene observation into a sequence of executable skills [1]–[4]. A large and fast-growing body of work validates these planners in simulation: scripted physics engines such as CoppeliaSim [5] or MuJoCo [6], or, further still, purely symbolic environments in which the “scene” is a hand-authored text description and perception is a proxy that returns whatever the experiment designer wrote down [7]. Symbolic and scripted validation is valuable – it is fast, reproducible, and lets a researcher isolate the reasoning strategy from perception and control. But it has a structural blind spot: an object’s height, its grasp width, and the vertical distance a gripper must travel to release it safely are never actually *measured* in these environments. They are simply asserted, correctly, by construction.

On real hardware, none of these quantities are given. They must be computed, from noisy depth, from a calibrated but imperfect camera, or from a hand-authored scene file that may or may not match the physical table it describes. Over six development sessions deploying an LLM/VLM planning stack on a real UR5cb manipulator with an OnRobot RG2 gripper and a calibrated RGB-D camera, we observed a recurring and costly pattern: when a planning prompt asks the model to compute a piece of manipulation geometry itself – a release height, a stacking height, a grasp width – that computation is often wrong, in ways that are silent until the arm physically drops, crushes, or misses an object. The failures are not random. They cluster around a single, identifiable root cause: the model is asked to reason about a physical quantity that a sensor, not language, should determine.

This paper makes that observation precise and actionable. We diagnose six recurring geometry-grounding bugs from production hardware logs (§III), show that they share a common structural cause across two independently-implemented planning pipelines (a static-scene LLM planner and a camera-grounded VLM planner), and close five of them by removing geometric arithmetic from the model’s output entirely, replacing it with a deterministic grounding layer that computes the same quantities from paired pick/release actions and measured depth (§IV). The sixth – a multi-object placement collision – we diagnose but have not yet closed, and it turns out to be the dominant one: a 120-trial real-hardware benchmark (§V) shows it as the residual failure distribution once the other five are fixed, occurring in *both* the LLM and VLM pipelines regardless of which one grounded the scene – direct evidence that the failure tracks the geometry-grounding architecture, not the choice of grounding modality.

Our central argument is that this failure class is not a peculiarity of our codebase, but a structural consequence of validating LLM/VLM robot planners in simulation. We make this argument concretely against a direct, comparable prior work: Favali et al. [7] propose a rigorous taxonomy of task and environment complexity and a benchmarking framework for LLM-based robotic planning, evaluated in a symbolic simulator where ground truth is always exactly known. Their framework is, by design, incapable of producing the failure mode this paper documents, because their simulated agent never computes a grasp height that a physical sensor could contradict. We do not consider this a weakness of their study – their methodology is sound for the question it asks – but we argue that a geometry-grounding audit of

¹The authors are with [Department, University]. E-mail: matteo.nini@[domain].

the kind we perform here is a necessary complement to any symbolic or scripted benchmark before an LLM/VLM planner is deployed on real hardware.

The contributions of this paper are threefold: nosep

- A taxonomy of six recurring real-hardware geometry-grounding bugs, diagnosed from production debug logs across six development sessions on a real manipulator (§III).
- A deterministic grounding layer that closes five of the six, removing model-computed geometric arithmetic from the planning-critical path, together with an evaluation of its effect (§IV).
- A 120-trial real-hardware benchmark, methodologically anchored to a directly comparable symbolic-simulation study [7] and using a single unified model for both grounding modalities, showing that the sixth, still-open bug – multi-object placement collision – is the dominant residual failure in *both* arms, and what that implies for evaluating LLM/VLM robot planners before hardware deployment (§V).

II. RELATED WORK

LLM/VLM planning for robotics. Foundation models have been used to ground natural-language instructions in robotic affordances [3], to act as zero-shot planners [4], and to generate feasible motion plans directly from instructions [8]. A range of reasoning strategies has been proposed on top of these capabilities: reactive, single-step prompting; interleaved reasoning-and-acting (ReAct) [9]; iterative self-critique (Self-Refine) [10], including a long-horizon sequential-planning variant validated at ICRA [11]; search-based deliberation (Tree-of-Thoughts) [12]; and ensemble voting over independently sampled plans (chain-of-thought with self-consistency) [13], [14]. Recent surveys catalogue this space in detail [1], [2], [15]–[17]. Our contribution is orthogonal to this line of work: we do not propose a new reasoning strategy, and hold the reasoning strategy fixed in our evaluation (§V). Instead, we show that the geometric grounding surrounding *any* of these strategies – how a chosen action’s numeric parameters are computed – is a separate failure axis that persists regardless of which reasoning strategy produced the action.

End-to-end and vision-language-action approaches. An alternative architecture collapses perception, reasoning, and control into a single model, as in vision-language-action (VLA) systems [18], [19] or code-generating policies that couple perception directly to control primitives [20]–[23]. These approaches can, in principle, learn to compensate for geometric grounding errors end-to-end, at the cost of the diagnosability that a decoupled Sense-Plan-Act pipeline provides – when an end-to-end model fails, it is difficult to determine whether the failure was a misperception, a reasoning error, or a motor-control error [7]. We deliberately retain a decoupled architecture, precisely because it lets us isolate and name the geometry-grounding failure class this paper documents; a fully end-to-end system would not permit the diagnosis in §III at all, only its symptoms.

Simulation and sim-to-real transfer. The sim-to-real gap is a long-studied concern in robot learning, primarily for learned low-level control policies [24]. Physics simulators such as Gazebo [25], MuJoCo [6], and CoppeliaSim [5] narrow this gap for control by modeling contact dynamics and actuation limits [26]. The gap we document is different in kind: it does not concern whether a *learned policy* generalizes from simulated to real dynamics, but whether a *language-model-computed* geometric quantity – one that a symbolic benchmark never forces the model to compute against a contradicting sensor – is trustworthy at all. To our knowledge this failure class has not been previously isolated and named for LLM/VLM robot planners specifically.

Direct comparison. The closest prior work to this paper is Favali et al. [7], which introduces a task/environment complexity taxonomy (task length, goal specificity, affordance perception; affordance density and four classes of dynamic disturbance) and TS/TSR/AETS metrics for benchmarking LLM-based robotic planning across six reasoning strategies, evaluated in a symbolic simulator with exact ground truth. We adopt the same evaluation philosophy and, for §V, the same metric family, but on real hardware, where ground truth for safety and success is not automatically available (§V details the resulting semi-automatic adaptation). A companion systems paper from the same group, RoboReason Lab [27], provides a modular simulation testbed for the same class of reasoning strategies in CoppeliaSim; like [7], it never grounds a plan in measured depth or a calibrated camera. Both are simulation-only by design, and both are structurally unable to produce the geometry-grounding failures this paper diagnoses, because neither ever asks a model to compute a quantity a physical sensor could contradict.

III. GEOMETRY-GROUNDING FAILURE TAXONOMY

This section diagnoses six recurring geometry-grounding bugs, each found on real hardware and each traced to its root cause using the per-run debug artifacts (raw model response, config, and, in VLM mode, the captured camera frame) that every planning call in our system records. Table I summarizes all six; the remainder of this section develops the two most instructive of the five closed cases in full, since they recurred independently in two different planning pipelines and therefore best illustrate that the underlying pattern – not a one-off implementation slip – is structural, then treats the sixth, still-open bug on its own (§III-C), since it turns out to be the dominant failure mode in §V.

A. Case 1: top-vs-mid-body grasp/release mismatch

Our executor deliberately grasps objects mid-body rather than at their top surface: the pick descent is $d = \min(0.5h, d_{\max})$, where h is the object’s height and $d_{\max}$ is a clamp that prevents the gripper’s rigid mounting body from crashing into tall objects. Centering the grasp mid-body, rather than pinching at the very top edge, produces a more stable hold on typical rigid objects.

The paired *release* action, however, computes how far to lift the object before opening the

TABLE I
GEOMETRY-GROUNDING FAILURE TAXONOMY. “SENSOR/ACTUATOR REQUIRED” MARKS THE PHYSICAL PROPERTY THAT MAKES EACH BUG IMPOSSIBLE TO OBSERVE IN A SYMBOLIC OR SCRIPTED SIMULATOR.

Bug	Symptom and root cause	Sensor/actuator required
Top-vs-mid-body grasp/release mismatch	Release-lift height set to an object’s full height, implicitly assuming a top-surface grasp; the executor actually descends mid-body ($d = \min(0.5h, d_{\max})$). Net effect: a systematic release overshoot of roughly half the object’s height. Recurred independently in two pipelines (VLM mode, then LLM mode).	Real gripper kinematics (mid-body descent is a deliberate stability choice, not a simulator default).
Double-counted stacking height	In a VLM→LLM hybrid pipeline, a depth-measured true top surface had the VLM’s separate, non-depth-verified size estimate added on top of it a second time by the downstream LLM’s own stacking formula ($\text{position.z} + \text{size}[2]$).	Real depth measurement of an already-correct surface height.
Footprint- vs. point-distance target matching	A release genuinely inside a tray’s real bounding box, but far from its registered center point, was treated as having missed the target zone by a point-distance check.	Real target geometry with non-trivial spatial extent (tens of centimeters), not a point.
Grasp-width sampled from the silhouette edge	Grasp width derived by deprojecting a bounding box’s left/right edge pixels – exactly on the object’s silhouette boundary, the single worst location for valid depth (occlusion/parallax). One failed edge sample aborted an otherwise valid plan.	Real depth noise at object boundaries; does not exist for a rendered or symbolic bounding box.
Width-dependent TCP off-set	The gripper’s fingers pivot, so flange-to-contact distance is a function of how far apart they are commanded to open, not a constant – wrong for any object whose width diverges from a single calibrated offset.	Real gripper mechanism (finger kinematics), absent from a symbolic action space.
Multi-object sequential-placement collision (open)	Zone-aware spacing keeps a target zone’s <i>final</i> release positions from overlapping, but does not prevent the arm clipping an already-placed object en route to the next release, nor a released object settling into a neighbor from residual momentum or a few cm of height error. Dominant failure mode in a 120-trial re-evaluation: every non-empty operator note (22/22) describes a collision or a fallen/displaced object during a multi-object task, in both the LLM and VLM pipelines.	Real multi-body contact between two already-placed objects and the arm’s approach trajectory – absent from any single-object placement check, symbolic or real.

gripper – the `object.height` parameter, added to `release.position.z` by the executor at execution time – by assuming the object was grasped at its top. This assumption is wrong by construction whenever the pick descended mid-body, which is always. The result is a systematic release overshoot of approximately $0.5h$: released objects land, or are dropped from, a height proportional to half their own size, rather than fractions of a centimeter above the surface.

This bug was fixed once, in our VLM pipeline, by deriving `object.height` directly from measured depth ($h = \max(z_{\text{surf}} - z_{\text{top}}, h_{\min})$) rather than from a model-reported size. It then recurred, independently, in our LLM pipeline several weeks later, because that pipeline’s prompt still asked the model to compute `object.height` from its own scene-description arithmetic (`object.height = size[2]`), which is exactly the assumption the VLM-mode fix had already identified as wrong. Fixing the bug once, in one pipeline, did not fix it everywhere it existed. The eventual, general fix – described in §IV – is to compute this quantity deterministically for *every* pipeline, precisely because the underlying error is not a mistake any particular prompt makes, but a mismatch between what the model is asked to assume and what the physical gripper actually does.

B. Case 2: double-counted stacking height

Our static scene description uses the convention `position.z = base reference`, `size[2] = height`, so that `position.z + size[2]` reconstructs a target’s top surface – the formula every LLM-mode prompt is given for computing a stacking release height. In our VLM→LLM

hybrid pipeline, a VLM grounding call first deprojects a target’s pixel location to a real, depth-measured world coordinate, then hands the resulting scene off to the same LLM planner used in static-scene mode. An early version of this hybrid set the generated scene’s `target.position.z` directly to the depth-measured top surface – already correct, and requiring no further adjustment – while leaving the VLM’s separate, un-verified `size[2]` guess untouched. The downstream LLM planner then applied its normal stacking formula, adding the guessed height on top of an already-correct value, inflating every stack-on-target release by however wrong that guess happened to be.

The fix follows the same pattern as Case 1: rather than trying to correct the VLM’s size guess, we derive the target’s real height from depth ($h = \max(z_{\text{top}} - z_{\text{surf}}, 0)$), with no minimum floor, since a flat target zone marked directly on the table should legitimately read as having near-zero height) and reset `position.z` back to a base reference, so that the existing downstream formula reconstructs the true top surface instead of double-counting it. The quantity computed by sensor is inserted *into* the representation the model already expects, rather than trusted from the model’s own output.

C. Case 6 (open): multi-object sequential-placement collision

Unlike Cases 1–2, this bug is not closed by §IV. When a task places several objects into the same target zone in sequence – stacking three cubes on a tray, arranging four in a line on the table – each release’s *final* position is spaced to avoid overlapping the others already placed, by a grid layout clamped to the zone’s real bounds (§IV). This closes the

static version of the problem: no two final positions occupy the same footprint. It does not address the *dynamic* version: nothing constrains the arm’s trajectory while approaching or releasing the n -th object to avoid clipping the $(n - 1)$ -th, already sitting in the zone, and nothing constrains a released object’s small residual motion (momentum, a few cm of height error) from displacing a neighbor after release. Both are genuinely three-dimensional, time-dependent contact problems – not resolvable by correcting a single scalar the way Cases 1–2 were.

We identified this bug from the same per-run debug artifacts as Cases 1–2, but by the time it was isolated, it was already the dominant failure mode in the evaluation of §V (22 of 22 non-empty operator notes, in both the LLM and VLM pipelines), rather than something we could re-run in isolation and close before reporting results. We report it here as an identified, partially-mitigated (by the static spacing fix), **open** problem, not a claimed contribution – §VI returns to what closing it would require.

D. The common pattern

The six bugs in Table I share a structure: a quantity that is trivially, exactly correct in a symbolic or scripted simulator becomes, on real hardware, (a) a sensor measurement subject to depth noise and occlusion, (b) a function of real actuator kinematics that a text-based action schema has no natural way to express, or (c), for Case 6, a consequence of real multi-body contact between two physical objects sharing space over time. None of the three failure modes can occur in an environment where object geometry is authored by the experiment designer or perfectly known by the physics engine – which is precisely the setting in which LLM/VLM robot planners are most commonly benchmarked [7], [27].

IV. DETERMINISTIC GROUNDING LAYER

A. Design principle

The fix for five of the six bugs in Table I follows the same rule: *pull geometric arithmetic out of the model and into code wherever the answer is computable from a sensor measurement or a paired action already in the plan, and leave the model responsible only for the symbolic decision of what to do and where, not how much.* The model still chooses which object to pick, which zone to release into, and in what order; a deterministic post-processing layer then overwrites the specific numeric fields most exposed to the failure modes of §III, using only information already available at plan time (the paired pick action, measured depth, and the scene’s registered target geometry). The sixth bug (Case 6, §III-C) is not a scalar this rule can simply overwrite – it is a trajectory-level, time-dependent contact problem – and is addressed separately, and only partially, below.

B. Object and release height

Given a generated plan, `fix_object_height` recomputes every release step’s `object_height` as the true vertical distance from the paired pick’s actual grasp contact point to the table surface – $h - d$ in the notation of

§III-A – rather than accepting whatever height value the model reported. This directly closes Case 1: the release-lift amount is now derived from the *same* descent the executor actually commanded during the pick, which by construction cannot disagree with itself, instead of from a separate, independently-computed model output that could.

`fix_release_height` performs the analogous correction for the release *position*. Two design choices matter here. First, matching a release against the scene’s registered targets is done by **footprint containment** ($\text{position} \pm \text{size}/2$) rather than point-distance to a target’s registered center – Case 3 in Table I showed that point-distance matching misses valid placements inside a target zone that is tens of centimeters wide. Second, a release position that *exactly echoes* a target’s or object’s own registered coordinates is treated as intentional (a deliberate stack or placement) and left untouched; a position that merely *drifted* into a zone’s footprint without echoing it is nudged back outside, along whichever axis requires the smaller correction, but only when that zone’s surface sits meaningfully above the bare table – nudging away from the near-table-height generic “table” zone would otherwise push objects off the table’s edge trying to escape a zone that legitimately contains most of the workspace.

C. Width-aware TCP offset

Because the gripper’s fingers pivot, the flange-to-contact distance for a given commanded width is not constant. We measure this relationship at three representative widths (fully closed, mid-open, fully open) and interpolate piecewise-linearly between them at plan time, setting the executor’s tool transform per-pick from the plan’s reported grasp width and resetting it to a default mid-open value after release. Pick descent is additionally capped at (calibrated offset – clamp clearance), so that tall objects are gripped nearer their top rather than driving the gripper’s rigid mounting body into them – this closes Case 5.

D. Zone-aware spacing: a partial mitigation for Case 6

For a target zone that receives multiple releases in one plan, we lay out occupants on a grid centered on the zone’s anchor point and clamped to its registered bounds, falling back to an unbounded grid only when no known zone matches. This closes the *static* half of Case 6: final release positions inside a zone no longer overlap, and no longer overflow a zone smaller than the model’s own (unclamped) spacing guess would have produced. It does not close the *dynamic* half – the arm’s approach trajectory to the n -th release is not checked against the already-placed $(n - 1)$ -th object’s footprint, and a released object’s small residual motion is not constrained at all. §V reports this as still the dominant failure mode; §VI returns to what closing the dynamic half would require.

E. Best-effort grasp-width sampling

For the VLM pipeline, grasp width is estimated by de-projecting a detected bounding box’s left and right edge pixels to measure real-world distance. Sampling directly on

the object’s silhouette boundary is the single worst location for valid depth, since boundary pixels are the most exposed to occlusion and parallax error; a single failed sample previously aborted the entire batched deprojection call, crashing an otherwise valid plan over a parameter – grasp width – that does not affect whether the *pick* and *release* coordinates themselves are safe. We fixed this by (i) sampling slightly inset from the raw silhouette edge rather than on it, landing on the object’s surface instead of its boundary, and (ii) isolating width estimation into its own best-effort call that degrades to the model’s own width guess on failure instead of aborting the plan. This closes Case 4 without requiring width to be depth-verified in every case – an open limitation we return to in §VI.

F. Validation

Each fix in this section was developed and iterated against real, previously-failing production runs: the per-run debug artifacts our system records for every planning call (raw model response, active configuration, and, in VLM modes, the captured camera frame) let us replay the exact inputs that produced a given hardware failure and confirm that the corrected pipeline now produces the hand-derived expected geometry for that same input, rather than validating only against synthetic test cases. This retrospective replay, together with a unit-level regression suite (45/45 passing across all reasoning-method implementations at the time of writing), gives us confidence that each fix addresses its diagnosed root cause. **A live, controlled hardware A/B ablation** – re-running the task conditions that originally surfaced these bugs with the deterministic grounding layer enabled versus disabled, and reporting the resulting effect on task safety and success directly – is designed but had not yet been executed at the time of writing; §V instead reports the residual failure distribution *with* the fix in place, and §VI discusses what the missing controlled comparison would add.

V. REAL-HARDWARE EVALUATION

A. Design

With the deterministic grounding layer of §IV in place, we ran a 120-trial real-hardware study comparing a static-scene LLM planner against a camera-grounded VLM planner, methodologically anchored to the task/environment complexity taxonomy and the TS/TSR/AETS metric family of Favali et al. [7], adapted to a physical rig. Six task conditions crossed three difficulty axes – task length, reference specificity, and affordance/perception load – each at an easy and a hard extreme, over a real scene of four color-distinguished cubes and a tray: `pp_easy/pp_hard` (pick-and-place, one vs. four objects, named directly), `sort_easy/sort_hard` (stacking and sorting), and `arith_easy/arith_hard` (the object to act on must be *computed* from a stated numeric value mapping rather than named directly – the one condition designed to isolate reasoning load from perception load). Each condition was run 10 times per model, for 120 trials total, with a single reasoning method (chain-of-thought with self-consistency) held fixed across all trials, since it is

reported as the most safety-robust strategy in [7] and avoids stacking reasoning-method variance on top of the LLM-vs-VLM comparison actually under test. Both the LLM and VLM planner use the same underlying model (`kimi-k2.6`) for text-only and image-grounded calls respectively – deliberately, so that grounding modality is the only variable isolated between arms, not a side effect of comparing two different models with different baseline capabilities.¹

B. Metrics, adapted for physical ground truth

Favali et al.’s TS (task safety), TSR (task success rate), and AETS (action efficiency on task success) metrics are fully automatic in a symbolic simulator, where whether an action “damages the environment” or a sub-task “completes” is always exactly derivable from simulator state. Our real UR5cb has no collision or force-fault sensor anywhere in its control path, and no vision-based verification of the final scene state. We therefore adapt the same three metrics to a **semi-automatic** protocol: everything the system can know automatically (the command, configuration, generated plan, and steps actually executed) is logged by the debug harness of §IV for every trial; the operator supplies exactly two numbers immediately after each trial – was it safe, and how many sub-tasks completed correctly – from which TS, TSR, and AETS are computed identically to [7]’s definitions. This adaptation is itself a consequence of the paper’s central argument: full automation of these metrics is only possible because a symbolic simulator’s world state is always exactly known, which is precisely the property real hardware does not have.

At $n = 10$ per cell, point estimates alone invite exactly the kind of pushback a reviewer should raise, so every reported TS% and TSR% below carries a 95% Wilson score interval (TS is an exact per-trial proportion; TSR pools successes over $\text{sub_tasks_required} \times n$). One comparison was declared before any data was collected, not chosen afterward to fit the numbers: a two-sided Fisher’s exact test, LLM vs. VLM, on `arith_hard` full-trial success, at $\alpha = 0.05$ – binarized per trial rather than pooled across a trial’s four sub-tasks, since those are not independent draws (one wrong decision tends to misplace every object in the trial together, not each independently). Everything else is reported descriptively only.

C. Results

Table II reports the headline result. Both models operate under the same deterministic grounding layer and the same underlying model, so the residual gap is neither a grasp-geometry gap (§IV applies uniformly to both pipelines) nor a model-capability gap – it is specifically attributable to grounding modality. Breaking results down by task condition (Figure 2) shows the gap concentrated on the two hardest multi-object conditions, and predominantly in *safety*, not

¹An earlier run used different models per modality; the VLM-specific model was subsequently withdrawn from its provider’s catalog, which forced a full re-run on a single unified model rather than a partial extension – see §VI for why this incidentally strengthens the comparison’s internal validity.

TABLE II

OVERALL RESULTS, 60 TRIALS PER MODEL (120 TOTAL), WITH THE DETERMINISTIC GROUNDING LAYER OF §IV IN PLACE.

Model	n	TS	TSR	AETS
LLM (kimi-k2.6)	60	98.3%	98.3%	0.1269
VLM (kimi-k2.6)	60	85.0%	89.2%	0.1149

Overall headline results — LLM vs VLM (n=60 per bar, 95% Wilson CI)

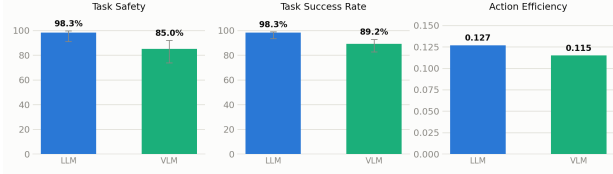


Fig. 1. Overall TS/TSR/AETS summary, LLM vs. VLM, with the deterministic grounding layer in place.

success rate: TS is 100% for both models on every easy condition and on pp_hard/sort_hard (LLM), but drops to 50% (VLM, sort_hard) and 60%/90% (VLM/LLM, arith_hard) on the two conditions with the most simultaneous in-zone objects. Neither model shows the sharp *success-rate* collapse a July pilot with different, modality-specific models had found on the arithmetic conditions specifically (30%/25% VLM TSR against 100%/97% LLM) – with a single unified model, arith_easy is 100%/100% and arith_hard is 90%/85% (TSR, LLM/VLM), a real but modest gap, not a collapse. §V-D returns to why. The one pre-registered comparison – Fisher’s exact test, LLM vs. VLM, arith_hard full-trial success – shows no detectable difference at this sample size (LLM 6/10, VLM 4/10, $p = 0.66$), consistent with a real but modest gap that $n = 10$ per cell cannot resolve to significance, not with an absence of one.

D. The failure mode is computation, not perception

Reading all 22 non-empty operator failure notes from the 120 trials, rather than trusting the aggregate numbers alone, was the key analysis step, as it was in an earlier pilot on this same task matrix. Every one of the 22 notes describes either a collision between objects (10) or an execution/grasp failure – a slipped grasp, a release too high, a dropped object (12) – during a multi-object task (pp_hard, sort_hard, arith_hard). **Zero** notes describe a wrong object chosen for reasoning or value-mapping reasons, and zero describe an incomplete enumeration (Figure 3). This happens in **both** arms: the LLM contributes 4 of the 22 notes (all on arith_hard), the VLM the remaining 18, spread across all three multi-object conditions roughly in proportion to their trial counts – the failure recurs regardless of which planner grounded the scene, direct confirmation that Case 6 (§III-C) is this taxonomy’s most consequential open item, not a peculiarity of one pipeline.

Unlike the closed bugs of §III, this result is not a second-order instance of the paper’s central claim – it is a direct one.

Safety and success rate by task — LLM vs VLM (n=10 per bar, 95% Wilson CI)

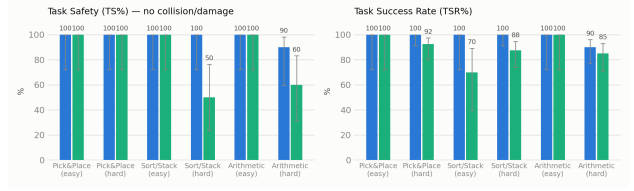


Fig. 2. Task safety and task success rate by task condition, with 95% Wilson CI.

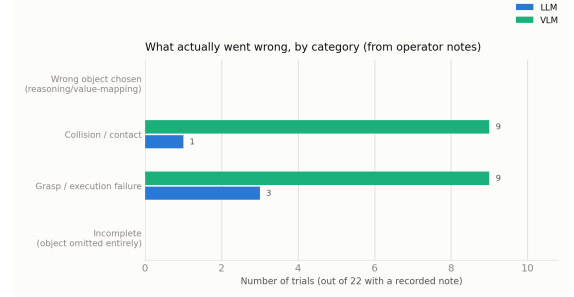


Fig. 3. Failure-mode breakdown, derived from the 22 non-empty operator notes out of 120 trials.

Case 6 is itself one of the six geometry-grounding bugs of Table I; this evaluation is what identifies it as the dominant one once the other five are closed.

A note on an earlier finding. A pilot run on this same task matrix, using different, modality-specific models (not the single kimi-k2.6 used here), found a different dominant category: 13 of 31 notes read as a wrong-object choice concentrated specifically in the arithmetic condition, consistent with an off-by-one error in value-mapping reasoning under a stated numeric mapping. That finding does not replicate here – zero such notes this cycle, on every condition. We read the simplest explanation as the correct one: it was a property of the earlier VLM model specifically, not of VLM grounding in general, and does not survive a model change. We report it for the record and as a caution against generalizing a single-model finding to “VLM grounding” as a category, not as a result that still stands.

VI. DISCUSSION AND LIMITATIONS

Implication for evaluation practice. If the argument of this paper is correct, symbolic and scripted simulation is necessary but not sufficient for validating an LLM/VLM robot planner: it can validate reasoning strategy, task decomposition, and high-level correctness, but by construction cannot exercise the geometry-grounding failure class of §III, because it never asks the model to compute a quantity a physical sensor could contradict. We suggest that a geometry-grounding audit – identifying every numeric quantity a planning prompt asks the model to compute itself, and asking whether a sensor or a paired action already determines that quantity more reliably – belongs alongside task-complexity benchmarking as a standard step before hardware deployment, independent of which reasoning strategy or foundation

model is used.

What the missing ablation would add. §IV-F is explicit that our deterministic grounding layer is validated retrospectively, against replayed real-hardware failures and a unit regression suite, rather than through a live controlled A/B comparison on hardware. A live ablation – re-running the task conditions that originally surfaced these bugs with the grounding layer enabled versus disabled – would let us report a quantitative safety/success delta attributable to the fix specifically, rather than inferring its necessity from the failures it was designed to close. Two scopes are designed but neither is run at the time of writing: a restricted version (LLM-only, the two release-geometry fixes specifically, roughly half a day including trials) and a complete one (both modalities, the full taxonomy including the safety-critical TCP-offset/grasp-width path, roughly two days). We flag the restricted version as the most direct next step, since it targets the mechanism the evaluation of §V actually implicates.

Case 6 remains the largest open item. Everything else in this paper is either closed (five of six bugs) or bounded by a controlled comparison we can eventually run (the ablation above). Case 6 is neither: it is a trajectory-level, multi-body contact problem, and §IV-D’s static-spacing mitigation, while a genuine improvement over an unbounded layout, does not touch the dynamic half of the problem at all. Closing it plausibly needs one of (a) a motion-planning-level check of the approach trajectory against already-placed objects’ known footprints, not just their final target positions, or (b) a settle-time verification step after each release, before the next pick begins, so a displaced neighbor is caught and corrected rather than compounding into the next release. We have not implemented or evaluated either.

Generalizability. Our evidence comes from one manipulator (UR5cb), one gripper (OnRobot RG2), one camera rig, and one reasoning strategy. Deliberately, it comes from a *single* underlying model (kimi-k2.6) used for both the LLM and VLM arm, rather than one model per modality – an earlier configuration with modality-specific models is what originally produced the now-non-replicating value-mapping-reasoning finding of §V-D, which in hindsight looks like it was at least partly a model-identity confound rather than a pure grounding-modality effect. We do not claim that the specific six bugs of Table I recur verbatim on different hardware; we claim that the *structural pattern* – geometry computed by a model rather than measured or derived from a paired action, and multiple real bodies sharing space over time – is a property of the architecture (asking an LLM/VLM to output numeric geometry at all, and placing more than one object in a shared zone), not of this particular robot, and should be audited for on any comparable deployment.

Grasp width remains unverified. Of the quantities in §IV, grasp width is the one our system still cannot cross-check against depth in every case: a single top-down depth sample determines an object’s height unambiguously, but not its left-right extent. Our best-effort edge-inset sampling (§IV) reduces, but does not eliminate, this gap; a depth-informed rectification of the camera view before grounding is a natural

next step we have not yet completed.

Reliability of operator-supplied labels. Our semi-automatic evaluation protocol (§V) depends on a single operator’s real-time judgment of safety and sub-task completion for each trial. We did not measure inter-rater reliability with a second, blinded annotator on a subsample of trials, which would strengthen confidence in the reported TS/TSR numbers independent of the geometry-grounding argument this paper makes. This is not a hypothetical risk: an early pass over this cycle’s data marked every trial safe regardless of outcome, caught only by cross-checking the safety label against the free-text operator note recorded for the same trial, and corrected before the numbers in §V were computed. We report the incident as a concrete illustration of exactly the reliability concern this paragraph raises, not as a suggestion that it recurs undetected elsewhere in this dataset.

VII. CONCLUSION

We diagnosed a recurring class of geometry-grounding failures in LLM/VLM robot planning, drawn from six sessions of real-hardware deployment, and showed that its structural cause – a model computing a physical quantity that a sensor, a paired action, or real multi-body contact already determines – is invisible to symbolic and scripted simulation by construction, including a directly comparable prior taxonomy-driven benchmark. Replacing model-computed geometry with a deterministic grounding layer closes five of the six bugs we diagnosed; a 120-trial real-hardware evaluation with that layer in place, using a single unified model for both grounding modalities, shows the sixth – a multi-object placement collision – as the dominant residual failure, occurring in both the LLM and VLM pipelines regardless of which one grounded the scene. We argue that a geometry-grounding audit is a necessary complement to task-complexity benchmarking for any LLM/VLM planner headed to physical hardware, and leave closing the sixth bug’s dynamic, trajectory-level half, a controlled hardware ablation isolating the grounding layer’s quantitative effect, and a depth-verified treatment of grasp width, as the most direct next steps.

REFERENCES

- [1] Y. Kim, D. Kim, J. Choi, J. Park, N. Oh, and D. Park, “A survey on integration of large language models with intelligent robots,” *Intelligent Service Robotics*, vol. 17, no. 5, pp. 1091–1107, 2024.
- [2] J. Wang, E. Shi, H. Hu, C. Ma, Y. Liu, X. Wang, Y. Yao, X. Liu, B. Ge, and S. Zhang, “Large language models for robotics: Opportunities, challenges, and perspectives,” *Journal of Automation and Intelligence*, vol. 4, no. 1, pp. 52–64, 2025.
- [3] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, *et al.*, “Do as I can, not as I say: Grounding language in robotic affordances,” *arXiv preprint arXiv:2204.01691*, 2022.
- [4] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, “Language models as zero-shot planners: Extracting actionable knowledge for embodied agents,” in *International Conference on Machine Learning*, pp. 9118–9147, PMLR, 2022.
- [5] E. Rohmer, S. P. N. Singh, and M. Freese, “CoppeliaSim (formerly V-REP): A versatile and scalable robot simulation framework,” in *Proc. of The International Conference on Intelligent Robots and Systems (IROS)*, 2013. www.coppeliarobotics.com.

- [6] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 5026–5033, IEEE, 2012.
- [7] F. Favali, L. Sabattini, and V. Villani, "LLM-based reasoning for robotic planning: Robustness to task and environmental complexity," *Robotics and Autonomous Systems*, 2026. In press. TODO(authors): update volume/pages once assigned.
- [8] K. Lin, C. Agia, T. Migimatsu, M. Pavone, and J. Bohg, "Text2motion: From natural language instructions to feasible plans," *Autonomous Robots*, vol. 47, no. 8, pp. 1345–1365, 2023.
- [9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. R. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *The Eleventh International Conference on Learning Representations*, 2022.
- [10] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhunoye, Y. Yang, *et al.*, "Self-refine: Iterative refinement with self-feedback," *Advances in Neural Information Processing Systems*, vol. 36, pp. 46534–46594, 2023.
- [11] Z. Zhou, J. Song, K. Yao, Z. Shu, and L. Ma, "ISR-LLM: Iterative self-refined large language model for long-horizon sequential task planning," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 2081–2088, IEEE, 2024.
- [12] S. Yao, D. Yu, J. Zhao, I. Shafraan, T. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," *Advances in Neural Information Processing Systems*, vol. 36, pp. 11809–11822, 2023.
- [13] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in Neural Information Processing Systems*, vol. 35, pp. 24824–24837, 2022.
- [14] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," *arXiv preprint arXiv:2203.11171*, 2022.
- [15] X. Huang, W. Liu, X. Chen, X. Wang, H. Wang, D. Lian, Y. Wang, R. Tang, and E. Chen, "Understanding the planning of LLM agents: A survey," *CoRR*, 2024.
- [16] L. Berti, F. Giorgi, and G. Kasneci, "Emergent abilities in large language models: A survey," *arXiv preprint arXiv:2503.05788*, 2025.
- [17] A. Billard, A. Albu-Schaeffer, M. Beetz, W. Burgard, P. Corke, M. Ciocarlie, R. Dahiya, D. Kragic, K. Goldberg, Y. Nagai, *et al.*, "A roadmap for AI in robotics," *Nature Machine Intelligence*, vol. 7, no. 6, pp. 818–824, 2025.
- [18] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choremanski, T. Ding, D. Driess, A. Dubey, C. Finn, *et al.*, "RT-2: Vision-language-action models transfer web knowledge to robotic control," *arXiv preprint arXiv:2307.15818*, 2023.
- [19] D. Driess, F. Xia, M. S. M. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu, *et al.*, "PaLM-E: An embodied multimodal language model," in *International Conference on Machine Learning*, PMLR, 2023.
- [20] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, "Code as policies: Language model programs for embodied control," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 9493–9500, IEEE, 2023.
- [21] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, "VoxPoser: Composable 3D value maps for robotic manipulation with language models," *arXiv preprint arXiv:2307.05973*, 2023. Conference on Robot Learning (CoRL) 2023.
- [22] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, P. Sermanet, N. Brown, T. Jackson, L. Luu, S. Levine, K. Hausman, and B. Ichter, "Inner monologue: Embodied reasoning through planning with language models," *arXiv preprint arXiv:2207.05608*, 2022.
- [23] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, and A. Garg, "ProgPrompt: Generating situated robot task plans using large language models," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 11523–11530, IEEE, 2023.
- [24] W. Zhao, J. P. Queralta, and T. Westerlund, "Sim-to-real transfer in deep reinforcement learning for robotics: A survey," in *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, pp. 737–744, IEEE, 2020.
- [25] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," in *2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (IEEE Cat. No. 04CH37566)*, vol. 3, pp. 2149–2154, IEEE, 2004.
- [26] J. Collins, S. Chand, A. Vanderkop, and D. Howard, "A review of physics simulators for robotic applications," *IEEE Access*, vol. 9, pp. 51416–51431, 2021.
- [27] F. Favali, L. Sabattini, and V. Villani, "RoboReason Lab: An accessible simulation framework for foundation models in robotics." Manuscript, Department of Sciences and Methods for Engineering (DISMI), University of Modena and Reggio Emilia. Code: <https://github.com/filippofavali/RoboReason-Lab>. TODO(authors): update with final venue/identifier once available., 2026.